Task-22 :- Requirement Analysis

# 1. Introduction

### 1.1 Purpose

The purpose of this document is to describe the requirements and analysis for setting up a self-hosted GitLab Community Edition (CE) server and pushing a Flask Hello World application to a repository hosted on it.

This document is intended for DevOps engineers, system administrators, developers, and project evaluators who are involved in the setup, configuration, and validation of the GitLab CE server and repository management.

---

### 1.2 Scope

**In Scope:**

- Installation and configuration of GitLab CE on AWS EC2
- Configuration of required ports and access permissions
- Creation of a GitLab repository/project
- Creation of a Flask Hello World application
- Pushing Flask application code to GitLab repository
- Validation of Git operations and repository visibility

**Out of Scope:**

- CI/CD pipeline configuration
- Production deployment of Flask application
- High availability GitLab setup
- Advanced monitoring and logging

---

### 1.3 Definitions, Acronyms, and Abbreviations

| Term | Description |
|---|---|
| GitLab CE | Self-hosted Git repository management platform |
| EC2 | Amazon Elastic Compute Cloud |

| | |
|---|---|
| Flask | Python web framework |
| Git | Distributed version control system |
| Repository | Storage location for source code |
| SSH | Secure Shell protocol |
| HTTP | Hypertext Transfer Protocol |
| HTTPS | Secure Hypertext Transfer Protocol |

---

### 1.4 References

- GitLab Official Documentation – GitLab Docs
- AWS EC2 Documentation – https://docs.aws.amazon.com/ec2
- Flask Official Documentation – Welcome to Flask — Flask Documentation (3.1.x)
- Git Documentation – Git - Reference

---

### 1.5 Overview

This document describes the overall requirements and analysis for deploying GitLab CE and pushing a Flask Hello World application to a GitLab repository.
The document includes project overview, functional requirements, non-functional requirements, assumptions, dependencies, risk analysis, and glossary.

---

## 2. Project Overview

### 2.1 Project Objectives

The main objectives of this project are:

- To deploy GitLab CE on an AWS EC2 instance
- To configure GitLab CE for web access
- To create a new repository in GitLab
- To develop a Flask Hello World application
- To push the Flask code to GitLab repository
- To verify repository access and Git operations

---

### 2.2 Business Goals

The business goals of this project are:

- To maintain an internal source code repository
- To provide version control for development teams
- To improve collaboration between developers
- To ensure secure and centralized code storage
- To build a self-hosted repository management system

---

### 2.3 Background

ABC Technologies wants to implement a self-hosted Git-based repository management system using GitLab CE.
Instead of using external platforms, the organization wants full control over its repositories and codebase.
To validate the setup, a sample Flask Hello World application will be created and pushed to the GitLab repository.

---

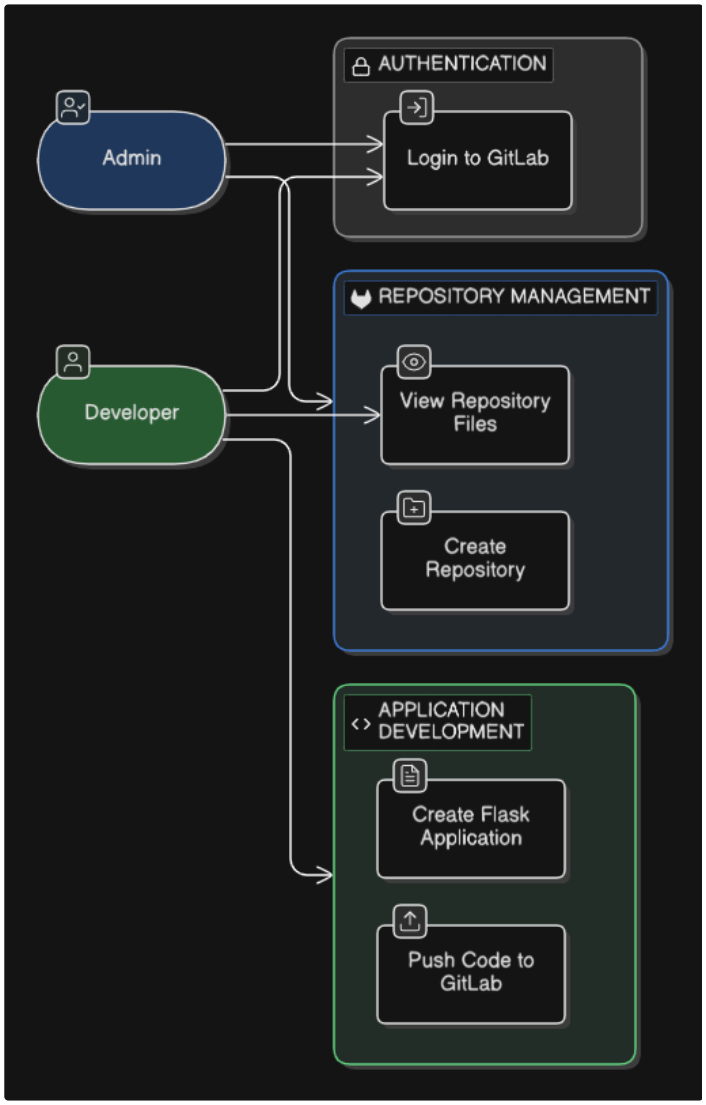
### 2.4 Constraints

| Constraint | Description |
|---|---|
| Time | The project must be completed within 3 hours |
| Budget | Limited AWS resources should be used |
| Infrastructure | Only one GitLab server will be deployed |
| Technology | Must use GitLab CE and Flask |
| Access | Requires proper security group configuration |

---

## 3. Functional Requirements

### 3.1 Use Case Diagram




Actors:

- Admin
- Developer

Use Cases:

- Login to GitLab
- Create Repository
- Create Flask Application
- Push Code to GitLab
- View Repository Files

## 3.2 Use Cases

---

### 3.2.1 Use Case 1: Login to GitLab

**Description:** User logs into the GitLab web interface.

**Actors:** Admin / Developer

**Preconditions:**

- GitLab server is installed and running
- GitLab URL is accessible

**Postconditions:**

- User successfully logged into GitLab dashboard

**Flow of Events:**

1. User opens GitLab URL in browser
2. Login page appears
3. User enters username and password
4. GitLab dashboard opens

**Alternative Flows:**

- Incorrect credentials → Error message displayed

---

### 3.2.2 Use Case 2: Create Repository

**Description:** Admin creates a new repository for Flask application.

**Actors:** Admin

**Preconditions:**

- Admin logged into GitLab

**Postconditions:**

- New repository created

**Flow of Events:**

1. Click on New Project
2. Enter project name
3. Select visibility level

4. Click Create Project

**Alternative Flows:**

- Project name already exists → Error message

---

### 3.2.3 Use Case 3: Create Flask Hello World Application

**Description:** Developer creates Flask Hello World application.

**Actors:** Developer

**Preconditions:**

- Python and Flask installed

**Postconditions:**

- Flask application created successfully

**Flow of Events:**

1. Create project folder
2. Create app.py file
3. Write Flask Hello World code
4. Run application locally

**Alternative Flows:**

- Flask not installed → Install Flask

---

### 3.2.4 Use Case 4: Push Code to GitLab Repository

**Description:** Developer pushes Flask code to GitLab repository.

**Actors:** Developer

**Preconditions:**

- Git installed
- Repository created in GitLab
- Remote repository configured

**Postconditions:**

- Code pushed successfully

**Flow of Events:**

1. Initialize Git repository
2. Add files to Git
3. Commit files
4. Add remote repository URL
5. Push code to GitLab

**Alternative Flows:**

- Authentication failure → Push rejected

---

### 3.2.5 Use Case 5: View Repository Files

**Description:** User verifies pushed code in GitLab repository.

**Actors:** Admin / Developer

**Preconditions:**

- Code pushed successfully

**Postconditions:**

- Files visible in GitLab repository

**Flow of Events:**

1. Open GitLab project
2. Navigate to repository
3. View Flask application files

**Alternative Flows:**

- Files not visible → Push failed

---

## 4. Non-Functional Requirements

### 4.1 Performance Requirements

- GitLab web interface should load within 5 seconds
- Git push operation should complete successfully without timeout
- Repository should be accessible at all times

---

### 4.2 Security Requirements

- HTTPS access for GitLab web interface
- SSH authentication for Git operations
- Secure AWS security group configuration
- Admin password protection

---

### 4.3 Usability Requirements

- GitLab interface should be user-friendly
- Repository creation process should be simple
- Clear navigation for repository and files

---

### 4.4 Reliability Requirements

- GitLab server uptime should be maintained
- Repository data should not be lost
- Git operations should work consistently

---

### 4.5 Compliance Requirements

- Follow AWS security best practices
- Use secure authentication methods
- Maintain access control for repositories

---

## 5. Assumptions and Dependencies

### 5.1 Assumptions

- AWS account is available
- Internet connection is available
- Basic knowledge of Git and Linux is available
- Python and Flask can be installed

---

### 5.2 Dependencies

- AWS EC2 instance
- GitLab installation packages

- Python and Flask
- Git version control system

---

## 6. Risk Analysis

### 6.1 Risk Identification

| Risk | Description |
|---|---|
| GitLab installation failure | Setup delay |
| Security group misconfiguration | GitLab not accessible |
| Git authentication issue | Code push failure |
| Flask dependency missing | Application not working |

---

### 6.2 Risk Mitigation

| Risk | Mitigation |
|---|---|
| Installation failure | Follow official GitLab documentation |
| Port access issue | Configure security groups properly |
| Git authentication | Use SSH keys |
| Flask dependency | Install required packages |

---

## 7. Glossary

| Term | Meaning |
|---|---|
| Git | Version control system |
| GitLab | Git repository hosting platform |
| Flask | Python web framework |
| Repository | Storage location for code |
| Commit | Save changes to repository |

## 8. Appendices

*(Insert Screenshots Below)*

- GitLab Dashboard Screenshot
- Repository Creation Screenshot
- Flask Code Screenshot
- Git Push Screenshot

---

## 9. Customer Acknowledgement

**Name:** _____

**Title:** _____

**Signature:** _____

**Date:** _____